

建设策略：全流程蓝图 vs 痛点闭环

文档类型： 建设策略
适用对象： 项目负责人 / 算法 / 软件 / IT
版本： v1.0
日期： 2026-07-08

1. 核心问题

开始着手建设时，是否需要先把完整算法开发流程所有功能罗列好？是先搭整体框架，还是围绕痛点做闭环？

结论

****先做“全局梳理 + 边界定义”，再围绕痛点做闭环。****
架构上有整体观，实施上做小闭环。

2. 是否需要先罗列完整流程？

要，但只需 L1/L2 粒度，不要穷尽所有细节。

第 0 步应输出 4 份文档（约 0.5~1 人月）：

输出物	内容
流程图	客户数据 → 最终交付全链路
功能树	拆到二级能力（Case 管理、推理执行、训练、转换、板测、发布）
接口清单	训练服务器 / 板端 / Docker 转换环境
优先级清单	MVP 范围、暂不做的、保留人工的

每个节点需明确：输入/输出、责任人（算法/软件/IT）、是否痛点节点。

3. 为什么不建议先做“大而全平台”

- 1. ****跨团队多****（IT、软件、算法），边界不清则接口反复改
- 2. ****流程知识未显式化****，很多规则还在人脑中，平台容易空心化
- 3. ****ROI 高度集中在两个痛点****，不应分散投入

4. 推荐路线：小框架 + 小闭环

最小公共骨架（5 个组件）

组件	作用
----	----

统一任务单/Case 单	全系统主键，关联问题与迭代
统一制品管理	数据、模型、转换产物、报告、文档
元数据存储	客户、芯片、版本、问题类型、状态、责任人
执行适配器	调用训练机 / Docker / 板端任务
报告与状态流转	任务进度、卡点、结论

两个痛点闭环

闭环	痛点	优先级
A	从拿到客户数据到明确下一步动作太慢	**先做**
B	模型训练好后同步给客户的闭环太长	后做

A 是入口瓶颈，入口不顺则后续自动化收益被抵消。

5. 闭环 A 概要

目标

客户数据 → 自动标准化 → 自动跑关键结果 → 自动差异报告 → 给出下一步建议 → 人确认

MVP 六项能力

- 1. badcase 入库规范
- 2. 自动数据整理与标准化
- 3. 自动跑浮点结果
- 4. 自动/半自动跑板端结果
- 5. 自动差异分析与分流建议
- 6. 自动报告 + 历史 case 检索

人月：5~7（轻量闭环版）

6. 闭环 B 概要

目标

模型入库 → 自动转换链路 → 自动编译/板测 → 自动汇总 → 人工审批 → 打包交付

MVP 六项能力

- 1. 模型登记与版本管理
- 2. 转换流水线（PTQ/QAT/预编译/导出）
- 3. 代码编译流水线
- 4. 板端测试流水线

- 5. 达标分析与审批
- 6. 自动生成交付材料

人月：4~6（轻量 MVP）

7. 综合人月估算

组成	人月
共享骨架	2~3
闭环 A	3~5
闭环 B	3~5
MVP 合计	**8~12**
较稳健版本	12~16
完整平台版	16~22

8. 推进顺序

阶段	内容	人月
0	流程蓝图 + MVP 范围定义	0.5~1
1	闭环 A（问题分诊）	5~7
2	闭环 B（模型交付）	4~6
3	规则化 + AI 助手增强	按需

9. 给内部的表述建议

不建议先做全功能平台，而建议先建设统一任务骨架，优先打通两个高价值闭环：第一闭环解决“客户数据进来后分诊太慢”；第二闭环解决“训练完成后交付链太长”。在可控投入下先压缩重复劳动和跨团队等待，再决定是否扩展为完整平台。